Audio Denoising

Problem Description :

Dataset Link :

Problem Description: Audio signals are prone to noise, and often these noises can interfere with important speech and music features, which can reduce their quality. Audio denoising is the process of removing unwanted noise from an audio signal while keeping the features that are important to human perception. This project aims to implement an audio denoising system that can remove unwanted noise from an audio signal, resulting in a high-quality output signal.

Dataset: There are several datasets available that can be used for audio denoising, such as the NOISEX-92 dataset, which is a collection of environmental noise samples, or the UrbanSound8K dataset, which is a collection of urban sound recordings. Additionally, it is possible to create your own noisy dataset by artificially adding noise to existing clean audio signals.

Objectives: The objective of this project is to create an audio denoising system that can automatically remove unwanted noise from an audio signal, while preserving the important features of the signal. The system should be able to handle a variety of different noise types and signal sources, and should be able to generalize well to new and unseen examples.

Deliverables: The deliverables for this project include a working audio denoising system, which can be used to remove noise from an audio signal, along with documentation and a report detailing the implementation and performance of the system.

Steps:

1. Load the audio dataset.
2. Preprocess the data by normalizing and reshaping the audio signals.

3. Create a noise reduction model, using techniques such as spectral subtraction, Wiener filtering, or deep learning models such as denoising autoencoders or convolutional neural networks.
4. Train the noise reduction model on the clean and noisy audio signals.
5. Evaluate the performance of the trained model on a test dataset, using metrics such as signal-to-noise ratio (SNR) or perceptual evaluation of speech quality (PESQ).
6. Use the trained model to denoise new audio signals and measure the denoising performance.

Note: The above steps are just a general framework, and the specific implementation may vary depending on the chosen dataset and noise reduction technique.

Dataset Link: NOISEX-92 dataset: https://datashare.is.ed.ac.uk/handle/10283/2651
UrbanSound8K dataset: https://urbansounddataset.weebly.com/

Code Explanation :

Here is the simple explanation for the code which is provided in the code.py file.

1. **Import required libraries:** The code starts by importing necessary libraries including "numpy", "librosa", "noisereduce", and "matplotlib".
2. **Load audio file:** The audio file is loaded using "librosa.load()" function, which returns a tuple containing audio data and sample rate.
3. **Plot audio file:** The original audio waveform is plotted using "librosa.display.waveplot()" function and "matplotlib.pyplot".
4. **Add noise to audio:** Another audio file containing noise is loaded and added to the original audio using "data + noise_data". The audio length of the noise file is also checked and shortened to the length of the original audio.
5. **Plot noisy audio:** The noisy audio waveform is plotted using the same method as in step 3.
6. **Apply noise reduction:** The "noisereduce.reduce_noise()" function is used to reduce the noise in the audio data. The function takes the audio clip data and noise clip data as input and returns the denoised audio data.
7. **Plot denoised audio:** The denoised audio waveform is plotted using the same method as in step 3.
8. **Save denoised audio to file:** The denoised audio data is saved to a new file using "librosa.output.write_wav()" function.

   The code thus applies noise reduction to the audio data and saves the denoised audio to a new file.

Future Work :

1. **Explore different types of noise reduction techniques**: In this step, you can explore different noise reduction techniques, including spectral subtraction, Wiener filtering, and adaptive filtering, and evaluate their performance. You can choose the most suitable technique for your audio denoising project.
2. **Build a machine learning model**: You can build a machine learning model to automatically learn the noise profile from the audio and apply appropriate noise reduction techniques. Deep learning models such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs) can be used for this purpose.
3. **Create a web application**: You can create a web application to provide the audio denoising service to users. The user can upload their audio files to the web application, and the application will return the denoised audio file.
4. **Develop an audio cleaning plugin**: You can develop an audio cleaning plugin for popular audio editing software such as Audacity, Adobe Audition, and Pro Tools. The plugin can provide the audio denoising service directly in the audio editing software, allowing users to clean up their audio files without leaving their favorite audio editor.
5. **Add support for multi-channel audio**: In this step, you can add support for multi-channel audio, allowing users to clean up their multi-channel audio files.

To implement these steps, here's a step-by-step guide:

1. **Explore different noise reduction techniques**: To explore different noise reduction techniques, you can start by reading research papers and blog posts on audio denoising. You can implement different techniques and evaluate their performance by measuring the signal-to-noise ratio (SNR) and other metrics.
2. **Build a machine learning model**: To build a machine learning model, you can start by collecting a dataset of audio files with different types of noise. You can use the dataset to train a deep learning model such as a CNN or RNN. You can use Python libraries such as TensorFlow or PyTorch to build your model.
3. **Create a web application**: To create a web application, you can use a web framework such as Flask or Django. You can use a frontend library such as React or Vue.js to create the user interface. You can use a cloud service such as AWS or GCP to host your web application.
4. **Develop an audio cleaning plugin**: To develop an audio cleaning plugin, you can use a plugin development kit such as VST, AAX, or AU. You can use a programming language such as C++ or JUCE to build your plugin. You can use a digital audio workstation (DAW) such as Pro Tools, Cubase, or Logic Pro to test your plugin.

5. **Add support for multi-channel audio**: To add support for multi-channel audio, you can modify your machine learning model and your denoising algorithm to handle multiple channels. You can also modify your web application and your audio cleaning plugin to support multi-channel audio.

Exercise :

Try to answers the following questions by yourself to check your understanding for this project. If stuck, detailed answers for the questions are also provided.

Here.